

基于OpenHarmony的智能点单系统

作品背景

随着智能终端和物联网的发展，智能餐饮管理系统逐渐成为智慧生活的重要组成部分。OpenHarmony 提供了轻量级内核 LiteOS-M，并支持任务管理（Task）、事件通信（Event）、Shell 命令注册等丰富的系统能力，非常适合作为嵌入式系统开发平台。

本作品基于 OpenHarmony LiteOS-M，设计并实现了一个智能点单系统。用户通过 Shell 输入菜单编号即可完成点单，系统利用事件通信机制将订单发送给不同厨师任务，实现"点单—处理—出菜"的完整流程，从而模拟现实餐厅中的订单处理过程。

整个系统重点展示了 OpenHarmony 多任务管理及任务间通信机制在实际场景中的应用。

功能特点

本系统主要实现了以下功能：

- 提供 Shell 命令 `menu` 实现命令行交互点餐；
- 内置 7 道菜品菜单供用户选择；
- 系统启动后自动创建两位厨师任务；
- 利用 Event 事件实现订单通知；
- 两位厨师分别负责不同菜品，提高处理效率；
- 实现订单处理、上菜全过程模拟；
- 支持非法编号检测及菜单重新显示。

设计思路

整个系统采用 **Shell → 点单模块 → Event事件 → 厨师任务** 的工作模式。

系统初始化时：

- 创建 Event 控制块；
- 创建两个 Chef Task；
- 注册 Shell 命令。

用户输入：

```
menu 3
```

后，系统完成以下流程：

① Shell解析命令

↓

② 判断输入是否合法

↓

③ 根据菜品编号查找事件标志位

↓

④ 调用

```
LOS_EventWrite()
```

发送事件

↓

⑤ 对应厨师任务

```
LOS_EventRead()
```

收到事件

↓

⑥ 模拟做菜

↓

⑦ 输出上菜结果

其中：

Chef1 负责：

- 番茄炒蛋

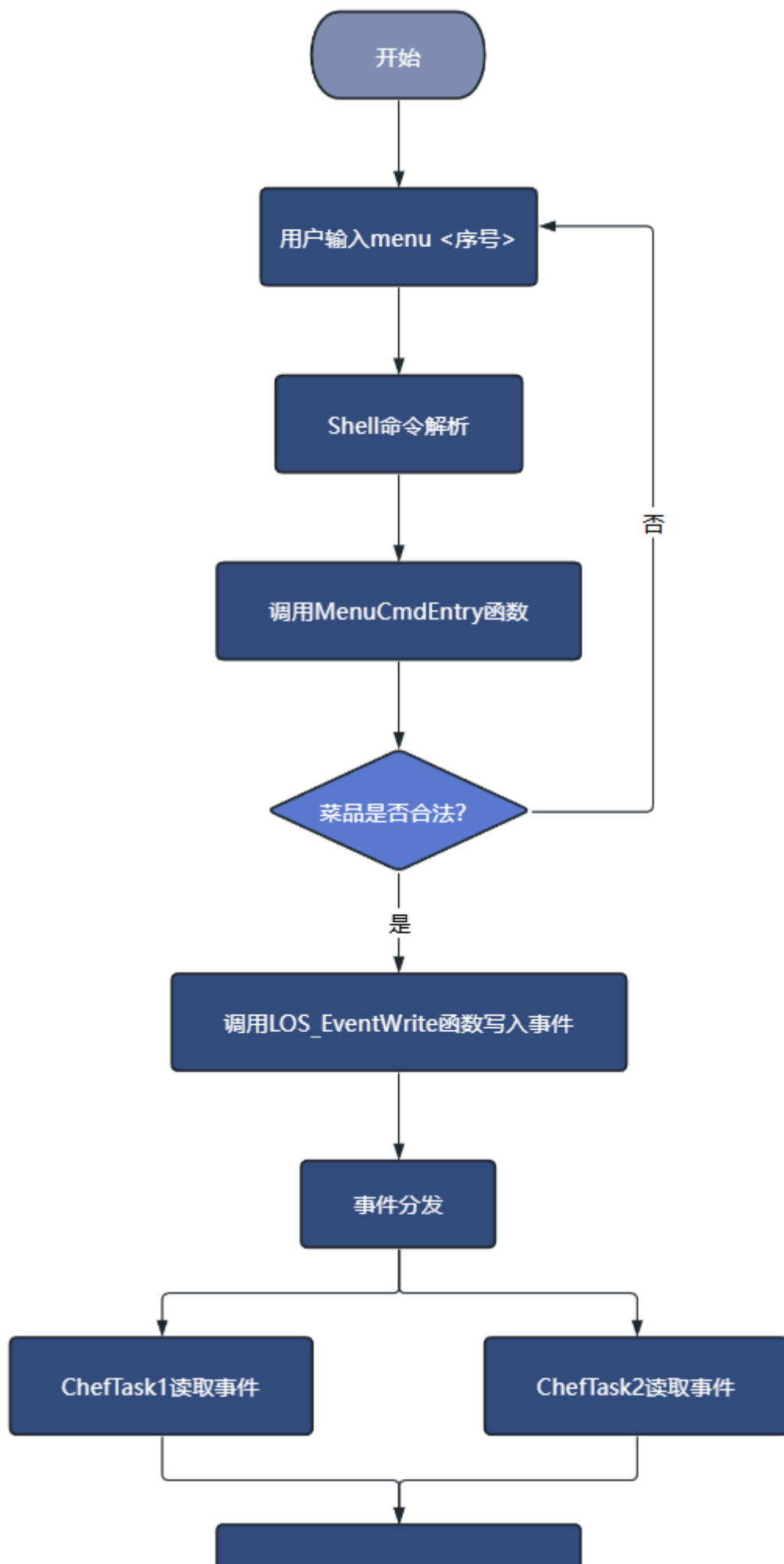
- 清蒸鱼
- 红烧肉
- 酸辣土豆丝

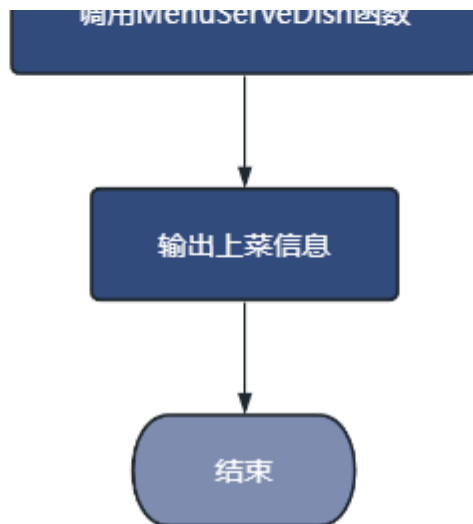
Chef2 负责：

- 青椒肉丝
- 麻婆豆腐
- 西红柿鸡蛋面

因此系统可以模拟多个任务分别处理不同订单。

软件框架图





实验步骤

(1) 创建菜单数据

建立菜品结构体：

```
typedef struct {
    UINT32 flag;
    const CHAR *name;
} MENU_DISH_S;
```

并建立菜单数组：

```
static const MENU_DISH_S g_menuDishTable[]
```

实现菜品编号与事件之间的一一对应。

(2) 初始化事件

调用

```
LOS_EventInit()
```

创建系统事件控制块。

所有订单均通过 Event 完成通信。

(3) 创建厨师任务

利用

```
LOS_TaskCreate()
```

创建两个厨师线程：

```
ChefTask1
```

```
ChefTask2
```

两个任务一直等待订单：

```
LOS_EventRead(...)
```

收到事件后立即处理。

(4) 注册 Shell 命令

调用osCmdReg(...)注册menu命令。

用户即可输入：

```
menu 5
```

完成点菜。

(5) 发送订单

MenuCmdEntry 中：

```
LOS_EventWrite()
```

根据菜单编号发送事件。

例如：

```
menu 2
```

发送：

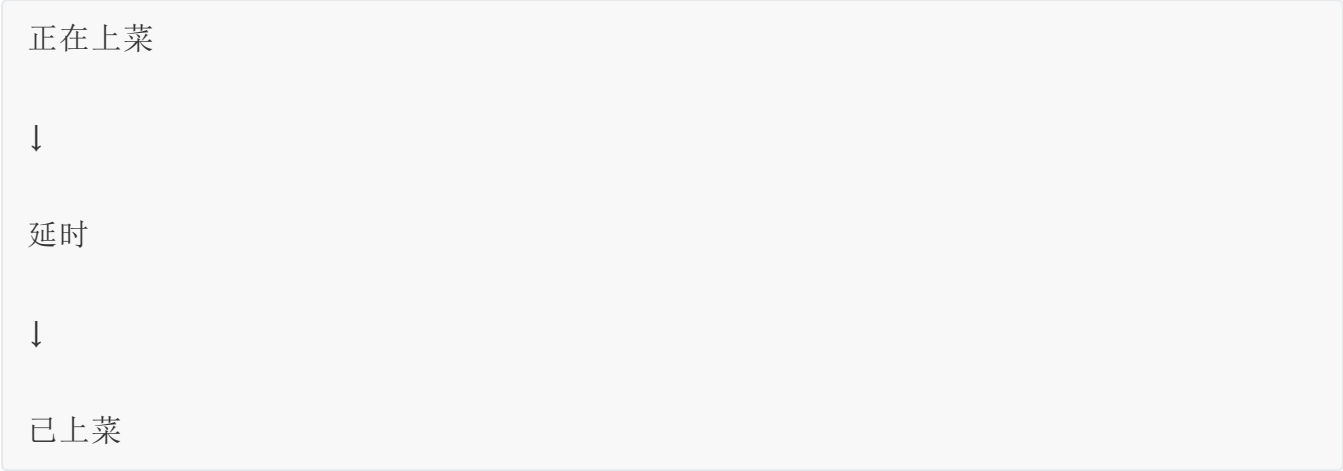
```
MENU_EVENT_DISH_2
```

(6) 厨师处理订单

厨师线程收到事件后调用：

```
MenuServeDish()
```

模拟：



完成整个订单流程。

实验结果

系统成功运行于 OpenHarmony LiteOS-M 平台。

实验过程中：

- 成功创建两个厨师任务；
- Shell 命令能够正常解析用户输入；
- Event 能够正确通知对应厨师；
- 不同菜品由对应厨师处理；
- 支持非法输入检测；
- 完整实现了"点单—处理—上菜"流程。

运行结果表明，系统能够稳定完成任务创建、事件通信以及命令交互，验证了 OpenHarmony Task 与 Event 机制在多任务协作场景中的可行性。